# Animation Design and Planning

Jetpack Compose Animation System

**Compose Animation Design Documentation**
30 Days of Fitness Application

**Student:** Priyanka
**Task:** Task 2 – Animation Design
**Technology:** Jetpack Compose + Material 3

# 1 Introduction

Animations play an important role in improving user experience in modern mobile applications. In Jetpack Compose, animations help users understand changes in UI state and provide visual feedback for interactions.

This document presents an animation planning strategy for the **30 Days of Fitness** Android application.

# 2 Compose Animation APIs

**animateFloatAsState**

This API animates a floating point value smoothly between states, commonly used for scale, rotation, or opacity transitions.

**animateColorAsState**

This function animates the transition between two colors when the state of the composable changes.

**AnimatedVisibility**

Controls the appearance or disappearance of composables with animation effects such as fade, slide, or expand.

**animateContentSize**

Automatically animates layout size changes when the content inside a composable changes.

**Transition**

A transition manages multiple animations simultaneously based on state changes.

# 3 Compose Animation Example

```
val color by animateColorAsState(
    targetValue = if (expanded)
        MaterialTheme.colorScheme.primary
    else
        MaterialTheme.colorScheme.surface
)
```

This code snippet demonstrates how the background color of a composable changes smoothly when the expanded state changes.

# 4 Selected Feature for Animation

The selected feature for animation in the application is the **Expandable Fitness Card**.

Each card represents a daily fitness activity. When the user taps on the card, the card expands to reveal additional content such as exercise details and images.

This animation helps present information gradually and improves the overall user interaction experience.

## 5 Application UI Screenshot

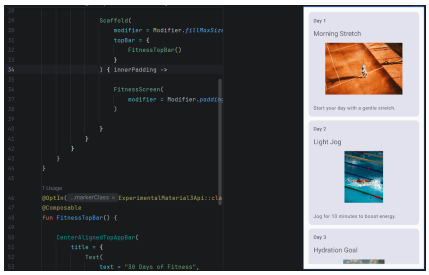


Figure 1: Main screen of the 30 Days of Fitness application showing fitness activity cards

The screenshot above shows the main screen of the application where daily fitness activities are displayed in card format. Each card contains the day number, activity title, image, and a short description.

These cards are the primary UI components where animation will be applied to enhance interaction.

## 6 Animation Plan

| Scenario | Trigger | Animation Type | UI Elements | Duration |
|---|---|---|---|---|
| Card Expand | Tap Card | animateContentSize | Card Height, Text | 300ms |
| Favorite Toggle | Tap Icon | animateColorAsState | Favorite Icon | 200ms |
| Image Reveal | Card Expanded | AnimatedVisibility | Image Component | 250ms |

## 7 Animation Flowchart

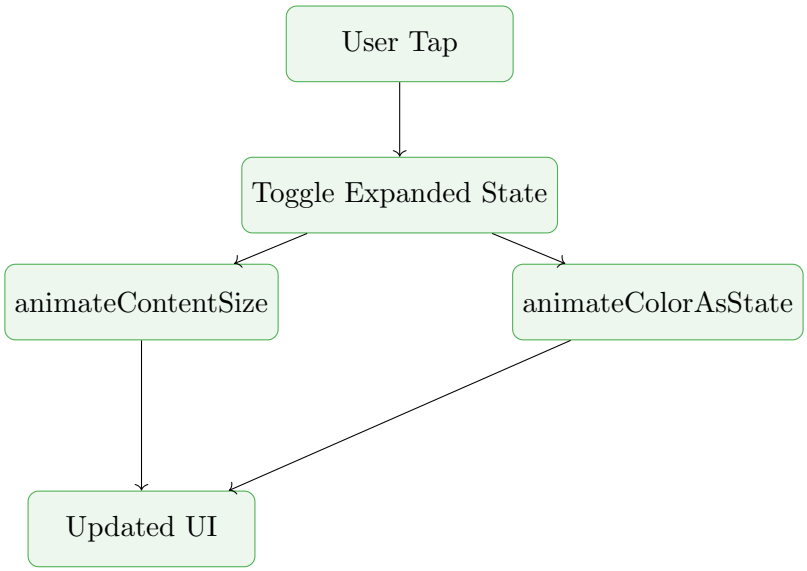


Figure 2: Animation Trigger Flow

## 8 Storyboard of Animation

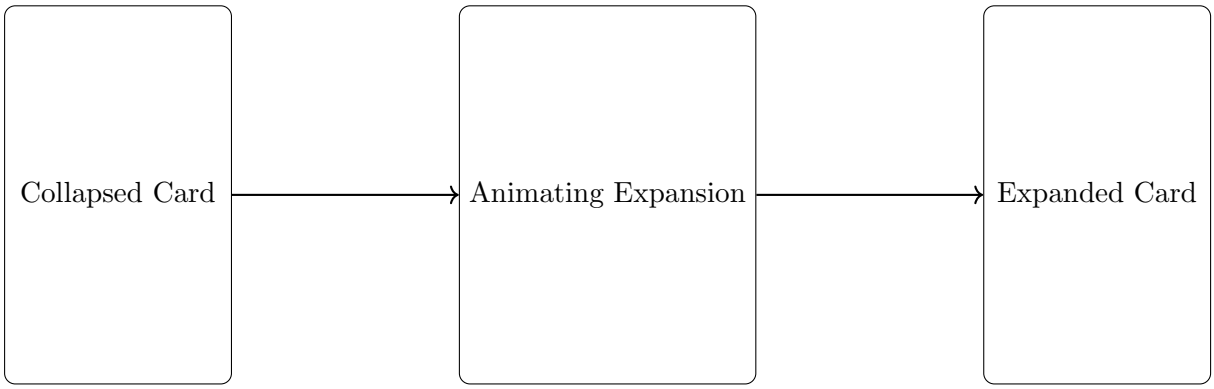


Figure 3: Storyboard of Card Expansion Animation

## 9 Performance Considerations

- Avoid heavy animations inside large LazyColumn lists to maintain smooth scrolling performance.
- Use lightweight animations such as animateContentSize instead of complex custom animations.

## 10 Accessibility Considerations

- Provide an option to reduce or disable animations for users who prefer minimal motion.
- Ensure all content remains readable and accessible even when animations are disabled.

## 11 Reflection

Planning animations before implementation helps developers design clear and meaningful UI interactions. Proper animation planning ensures that motion is purposeful and does not negatively impact performance or accessibility.

Animations should only be used to communicate changes in state or enhance user understanding. Overusing animations can make an application feel slow or distracting.

Carefully designed animations create a smoother and more engaging user experience.